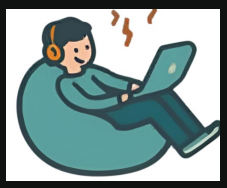

The Lazy Developer

19

MODULE

MCP: Supercharging Your AI Coding Workflow

Here we will dive into the Model Context Protocol (MCP) to manage context in AI-assisted coding. MCP can be configured within Cursor, Claude Code, and other AI tools.

Table of Contents

Introduction to MCP: The Universal Adapter for AI Tools

Imagine you just bought a brand-new phone charger, but it only works with one specific outlet type. Every time you travel to a different country, you need a completely different charger. Frustrating, right? Now imagine someone invents a single universal adapter that works with every outlet in the world. You plug it in, and it just works.

That is essentially what the Model Context Protocol (MCP) does for AI coding tools. It is a universal adapter that lets AI assistants like Cursor and Claude Code plug into external services, databases, APIs, and tools without needing a custom integration for each one.

Before MCP existed, every AI tool had to build its own bespoke connection to every service it wanted to talk to. Want your AI to query a database? Custom integration. Want it to read files from a specific cloud service? Another custom integration. Want it to interact with GitHub? Yet another one. It was like needing a different charger for every device you own.

MCP changes all of that.

What Is MCP, Really?

MCP stands for Model Context Protocol. It is an open standard created by Anthropic that defines a common way for AI tools to communicate with external services. Think of it as a shared language that both sides agree to speak.

Here is the key idea in one sentence: MCP lets your AI assistant access tools, data, and capabilities from the outside world in a standardized way.

Without MCP, if you wanted your AI to interact with your PostgreSQL database, someone would need to build a specific AI-to-PostgreSQL connector. And then if you also wanted say Claude Code to interact with that same database, someone would need to build a separate Claude-Code-to-PostgreSQL connector. Every combination of AI tool and external service required its own bridge.

With MCP, someone builds one PostgreSQL MCP server, and any AI tool that speaks MCP can use it. One bridge, many travelers.

The Client-Server Architecture

MCP uses a simple client-server model that you are probably already familiar with from web development:

- MCP Client = Your AI tool (Cursor, Claude Code, etc.). It is the one asking for things.
- MCP Server = The external service or capability being offered. It is the one providing things.

Here is how the conversation flows:

1. Your AI tool (the client) discovers what MCP servers are available
2. It checks what each server can do (what tools, resources, and prompts it offers)
3. When you ask the AI to do something that requires an external service, it calls the appropriate MCP server
4. The MCP server does the work and sends back the result
5. Your AI uses that result to help you

It is like walking into a restaurant. You (the client) look at the menu (discover capabilities), place an order (make a request), and the kitchen (the server) prepares your meal and brings it back.

What Can MCP Servers Provide?

MCP servers can offer three types of capabilities:

1. Tools (Actions)

These are things the AI can do. Think of them as verbs: run a database query, create a GitHub issue, search the web, take a screenshot.

2. Resources (Data)

These are things the AI can read. Think of them as nouns: file contents, database records, API responses, configuration data.

3. Prompts (Templates)

These are pre-built prompt templates that help the AI use the server effectively. Think of them as recipes: "Here is the best way to ask me to do X."

Capability	What It Does	Example
Tools	Performs actions	Run a SQL query, create a file, post to an API
Resources	Provides data	Read a database schema, fetch file contents
Prompts	Offers templates	"Analyze this database table" template

---

Real-World MCP Servers You Should Know About

Here are some popular MCP servers that are already available and incredibly useful:

- Filesystem Server: Lets your AI read, write, and manage files on your computer. Useful for project scaffolding and file organization.

- PostgreSQL / Supabase Server: Lets your AI query databases directly. You can ask it to check data, run queries, or inspect your schema.
- GitHub Server: Lets your AI create issues, review pull requests, search repositories, and manage your GitHub workflow.
- Brave Search Server: Gives your AI the ability to search the web for up-to-date information.
- Puppeteer Server: Lets your AI control a web browser, take screenshots, and interact with web pages.
- Memory Server: Gives your AI a persistent memory that survives between sessions.

These are not theoretical. These are real servers you can install and start using today.

How MCP Works in Cursor

Setting up MCP in Cursor is straightforward:

1. Open AI Settings (gear icon or Cmd + , on Mac)
2. Navigate to the MCP section
3. Add your MCP server configuration
4. Cursor will discover the server's capabilities automatically
5. Start using the tools in your conversations with the AI

Once configured, you can reference MCP tools directly in your prompts. For example, if you have a database MCP server configured, you can simply ask Cursor to query your database, and it will use the MCP server to do so.

```
// Example: Cursor MCP configuration (in settings)
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/path/to/your/
project"]
    }
  }
}
```

How MCP Works in Claude Code

Claude Code supports MCP configuration at both the project level and the global level:

- Project-level: Add an .mcp.json file in your project root. This is great for project-specific servers that your whole team can share.
- Global-level: Configure in your Claude Code settings for servers you want available everywhere.

```
// Example: .mcp.json in your project root
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres", "postgresql://
localhost:5432/m...
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "your-token-here"
      }
    }
  }
}
```

When Claude Code starts, it automatically connects to any configured MCP servers and can use their tools during your session.

Building Without MCP vs. With MCP

Let us compare the experience of working on a typical project task with and without MCP:

Task	Without MCP	With MCP
Check database schema	Open a separate DB client, run queries manually, copy results back to AI	Ask AI directly: "Show me the users table schema"
Create a GitHub issue	Switch to browser, navigate to GitHub, fill out the form	Tell AI: "Create an issue for this bug" and it does it
Search for documentation	Open browser, search, copy-paste relevant info back	AI searches the web and includes results in its response
Read project files	AI can only see files you explicitly open or paste	AI can browse your filesystem and read what it needs
Test a web page	Manually open browser, click around, describe what you see	AI launches a browser, navigates, takes screenshots
Query production data	Open DB client, write query, run it, interpret results	Ask AI: "How many users signed up this week?"

The pattern is clear: MCP eliminates the constant context-switching between your AI tool and other applications. Everything stays in one place.

Why MCP Matters for You

If you are using AI tools to build applications (which is exactly what this course is about), MCP is a game-changer for three reasons:

1. Less context-switching: You do not need to bounce between your AI assistant and a dozen other tools. The AI can reach out to those tools directly.
2. Better AI responses: When your AI has direct access to your actual database, your actual codebase, and your actual project tools, its suggestions are grounded in reality, not guesses.

3. Growing ecosystem: Because MCP is an open standard, anyone can build MCP servers. The ecosystem is expanding rapidly, which means more tools become available to your AI over time without you doing anything.

AI Prompt Examples to Try

Here are some prompts you can experiment with once you have MCP configured:

> *With a filesystem MCP server:*
> *"Look at my project structure and suggest how to organize my components better."*

> *With a database MCP server:*
> *"Query the users table and tell me how many accounts were created in the last 30 days."*

> *With a GitHub MCP server:*
> *"Create a GitHub issue titled 'Fix login redirect bug' with a description of the problem we just discussed."*

> *General MCP discovery:*
> *"What MCP tools do you have available right now? List them and explain what each one does."*

> *Combining MCP with development:*
> *"Check my database schema, then look at my API routes, and tell me if there are any mismatches between the two."*

Key Takeaways

- MCP is a universal adapter that lets AI tools connect to external services using a standard protocol, eliminating the need for custom integrations.
- The architecture is client-server: your AI tool is the client, external services are MCP servers.
- MCP servers provide three things: tools (actions), resources (data), and prompts (templates).
- Popular MCP servers already exist for databases, GitHub, file systems, web search, and browser automation.
- Both Cursor and Claude Code support MCP, with configuration through settings or project-level config files.
- MCP reduces context-switching by letting your AI access external tools directly instead of you copying information back and forth.
- The ecosystem is growing fast because MCP is an open standard that anyone can build on.

In the next section, we will get hands-on and walk through setting up your first MCP server step by step.

Implementing MCP: Getting Your Hands Dirty

Alright, now that you understand what MCP is and why it matters, it is time to actually set it up. By the end of this section, you will have configured MCP servers in both Cursor and Claude Code, know where to find new servers, and understand the security implications of giving your AI access to external tools.

> *Note: Don't add lots of MCP servers for the sake of it. Some analysis has shown that some coding environment pre-load information from them and take up some context. Use what's most important for that session, and dsiable it if you don't need it. Think of it like turning on and off the appropriate supporting information so you only use what's needed.*

Setting Up Your First MCP Server in Cursor

We will start with one of the simplest and most useful MCP servers: the filesystem server. This server lets your AI read and write files in a directory you specify.

Step 1: Open Cursor Settings

Click the gear icon in the bottom-left corner of Cursor, or press Cmd + , (Mac) or Ctrl + , (Windows/Linux). Navigate to the MCP section in the settings panel.

Step 2: Add an MCP Server

You will see an option to add a new MCP server. Click it, and you will need to provide a configuration. For the filesystem server, use this:

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/path/to/your/project"
      ]
    }
  }
}
```

Replace /path/to/your/project with the actual path to your project directory. On Mac, that might look like /Users/yourname/projects/my-app. On Windows, something like C:\\Users\\yourname\\projects\\my-app.

Step 3: Verify the Connection

Once you save the configuration, Cursor will attempt to start the MCP server. You should see a green indicator or a confirmation message. If something goes wrong, check the Cursor output panel for error messages.

Step 4: Test It Out

Open a new chat in Cursor and try a prompt like:

> *"What files are in my project root directory?"*

If the filesystem MCP server is working, the AI will use it to list your files instead of guessing.

Setting Up MCP in Claude Code

Claude Code gives you two ways to configure MCP servers:

Option 1: Project-Level Configuration (Recommended for Teams)

Create a file called .mcp.json in your project root:

```
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-postgres",
        "postgresql://localhost:5432/mydb"
      ]
    }
  }
}
```

This file can be committed to your repository (just be careful not to include secrets directly in it, more on security later). When anyone on your team uses Claude Code in this project, they will automatically have access to the same MCP servers.

Option 2: Global Configuration

For servers you want available in every project, you can configure them globally through Claude Code settings. Use the /mcp command within Claude Code to manage your MCP server configurations:

```
# Add a server using Claude Code CLI
claude mcp add github -- npx -y @modelcontextprotocol/server-github
```

This adds the GitHub MCP server to your global configuration so it is available in every Claude Code session.

Option 3: Claude Desktop Configuration

If you are using Claude Desktop, you can configure MCP servers in the claude_desktop_config.json file:

```
// macOS: ~/Library/Application Support/Claude/claude_desktop_config.json
// Windows: %APPDATA%\\Claude\\claude_desktop_config.json
{
  "mcpServers": {
    "brave-search": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-brave-search"],
      "env": {
        "BRAVE_API_KEY": "your-api-key-here"
      }
    }
  }
}
```

After saving, restart Claude Desktop for the changes to take effect.

Popular MCP Servers You Should Try

Here is a curated list of MCP servers that are particularly useful for the kind of development we cover in this course:

MCP Server	What It Does	Best For	Install Command
Filesystem	Read/write files and directories	Project exploration, file management	npx @modelcontextprotocol/server-filesystem /path
PostgreSQL	Query and manage PostgreSQL databases	Database inspection, data analysis	npx @modelcontextprotocol/server-postgres <conn-string>
GitHub	Manage repos, issues, PRs, and code search	Project management, code review	npx @modelcontextprotocol/server-github
Brave Search	Search the web using Brave Search API	Finding documentation, current info	npx @modelcontextprotocol/server-brave-search
Puppeteer	Control a headless browser	Testing, screenshots, web scraping	npx @modelcontextprotocol/server-puppeteer
Memory	Persistent key-value storage for AI	Remembering context between sessions	npx @modelcontextprotocol/server-memory
SQLite	Query and manage SQLite databases	Local database work, prototyping	npx @modelcontextprotocol/server-sqlite
Fetch	Make HTTP requests to any URL	API testing, fetching web content	npx @modelcontextprotocol/server-fetch

You do not need all of these at once. Start with the filesystem server, then add others as you need them.

Where to Find MCP Servers

The MCP ecosystem is growing quickly. Here are the best places to discover new servers:

1. Official MCP Server Repository

The main collection of official and community servers lives on GitHub at the Model Context Protocol organization. These are well-maintained and generally safe to use.

2. npm Registry

Many MCP servers are published as npm packages with the @modelcontextprotocol/ scope. You can search npm for them:

```
npm search @modelcontextprotocol
```

3. modelcontextprotocol.io

The official MCP website at modelcontextprotocol.io has documentation, server listings, and guides for getting started.

4. Community Directories

GitHub repositories like awesome-mcp-servers collect community-built servers. These can be incredibly useful but always review them before use since they are community-maintained.

Building a Custom MCP Server (High-Level Overview)

At some point, you might want an MCP server that does something specific to your workflow. The good news is that building a basic MCP server is not as intimidating as it sounds.

Here is the general process:

1. Choose Your Language

MCP servers can be built in any language, but the most common choices are TypeScript/Node.js and Python, since those have the best SDK support.

2. Install the SDK

```
# For TypeScript/Node.js
npm install @modelcontextprotocol/sdk

# For Python
pip install mcp
```

3. Define Your Server's Capabilities

You decide what tools, resources, and prompts your server will offer. For example, if you are building a server that integrates with your company's internal API:

- Tool: search_employees - search the employee directory
- Tool: get_project_status - check the status of an internal project
- Resource: company_handbook - provide the company handbook contents

4. Implement the Handlers

For each tool and resource, you write a handler function that does the actual work. The MCP SDK takes care of all the protocol communication for you.

5. Test and Configure

Run your server locally and add it to your Cursor or Claude Code configuration just like any other MCP server.

We will not build a full custom server in this course, but it is good to know that the option exists. Many developers use AI tools themselves to help generate MCP server code, which is beautifully meta.

Security Considerations

Giving your AI access to external tools is powerful, but it comes with responsibilities. Here are the key security considerations you need to be aware of:

What MCP Servers Can Access

An MCP server has access to whatever you configure it with. A filesystem server can read any file in the directory you point it to. A database server can run any query on the database you connect it to. A GitHub server with your personal access token can act on your behalf.

Rule of thumb: Only give MCP servers access to what they actually need.

Permission Models

Concern	Best Practice
File access	Point filesystem servers only at project directories, never at your entire home folder or system directories

Database access	Use read-only database credentials for MCP servers when possible
API tokens	Create tokens with minimal permissions (e.g., read-only GitHub tokens if you only need to search code)
Secrets in config	Never commit API keys or tokens to version control. Use environment variables instead
Network access	Be aware that some MCP servers make network requests. Only use servers from trusted sources

Environment Variables for Secrets

Instead of putting secrets directly in your MCP configuration, use environment variables:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

Then set the environment variable in your shell profile rather than in any committed file.

Approval Workflows

Both Cursor and Claude Code will typically ask for your approval before an MCP tool performs a significant action. Pay attention to these prompts. If the AI wants to delete files or modify your database, make sure that is actually what you intended.

Troubleshooting Common MCP Issues

Here are the most common problems you will encounter and how to fix them:

"Server failed to start"

Cause: Usually means the MCP server package is not installed or the command is wrong.

Fix: Try running the server command manually in your terminal to see the error:

```
npx -y @modelcontextprotocol/server-filesystem /your/path
```

If it fails, you may need to install Node.js or update npm.

"Connection refused" or "Server not responding"

Cause: The server started but crashed, or it is taking too long to initialize.

Fix: Check the server logs in your AI tool's output panel. Common causes include invalid configuration, missing environment variables, or network issues.

"Tool not found" when trying to use an MCP tool

Cause: The AI does not know about the MCP server, or the server has not been properly discovered.

Fix: Restart your AI tool after changing MCP configuration. Verify the server is listed in your MCP settings. Try asking the AI: "What MCP tools do you have available?"

Database connection failures

Cause: Wrong connection string, database not running, or network/firewall issues.

Fix: Test the connection string independently first:

```
psql "postgresql://localhost:5432/mydb"
```

Make sure your database is running and accessible.

Permission denied errors

Cause: The MCP server does not have permission to access the requested resource.

Fix: Check file permissions for filesystem servers. For database servers, verify the user has the necessary grants. For API servers, check that your token has the right scopes.

Putting It All Together: A Practical Workflow

Here is what a typical MCP-enhanced development session looks like:

1. Start your project in Cursor or Claude Code with MCP servers configured
2. Ask the AI to explore your project: "Look at my project structure and database schema"
3. Develop with context: The AI can check your database, read related files, and search your codebase while helping you build features
4. Manage your project: Create GitHub issues, review PRs, and search for solutions, all from within your AI conversation
5. Debug with real data: Instead of guessing, the AI can query your database and check actual application state

---

AI Prompt Examples for Working with MCP

Here are prompts to try once you have various MCP servers configured:

> *Filesystem + Development:*
> *"Look at my project files and create a new React component for a user settings page. Follow the same patterns you see in my existing components."*

> *Database + Debugging:*
> *"Query the orders table for the last 24 hours and check if there are any orders with a null status. If so, help me figure out what is causing it."*

> *GitHub + Project Management:*
> *"Look at the open issues in my repository and create a summary of what needs to be done for the next release."*

> *Brave Search + Research:*
> *"Search for the latest best practices for implementing JWT refresh tokens in Express.js and help me update my authentication middleware."*

> *Multiple servers combined:*
> *"Check my database schema, look at my current API routes, search GitHub issues for any related bugs, and suggest improvements to my user authentication flow."*

> *Troubleshooting MCP:*
> *"What MCP servers are currently connected? List each one and the tools it provides."*

Key Takeaways

- Start simple: Set up the filesystem MCP server first. It is the easiest to configure and immediately useful for project exploration.
- Cursor and Claude Code both support MCP but configure it differently. Cursor uses settings UI or JSON config, Claude Code uses .mcp.json files or CLI commands.
- Popular servers cover most needs: filesystem, databases, GitHub, web search, and browser automation handle the majority of development workflows.
- Find new servers on GitHub (modelcontextprotocol org), npm (@modelcontextprotocol scope), and modelcontextprotocol.io.
- Security is your responsibility: Only give MCP servers the minimum access they need. Use environment variables for secrets, read-only credentials when possible, and never commit tokens to version control.
- Troubleshooting is usually about basics: wrong paths, missing packages, invalid credentials, or needing to restart your AI tool after config changes.
- Building custom MCP servers is possible using the TypeScript or Python SDKs, and the AI itself can help you build them.

- MCP transforms your workflow from constant context-switching to a unified experience where your AI assistant has direct access to the tools and data it needs to help you effectively.